From LLM Reasoning to Autonomous AI Agents

Mengyue Yang

April 14, 2026

Outline

1. Introduction to LLMs and Planning

The Paradigm Shift

Large Language Models (LLMs) like GPT-4, Qwen2.5-Omni, DeepSeek-R1, and LLaMA have transformed AI from static text generation to dynamic reasoning.

- **Early LLMs:** Dependent on static pre-training data (prone to hallucination).
- **Current State:** Integration of planning, tool use, and environment interaction.
- **The Goal:** Highly autonomous AI systems (Agentic AI) capable of handling complex, multi-step workflows independently.

What is an Autonomous AI Agent?

- An AI Agent is a system that perceives its environment, makes decisions (plans), and takes actions to achieve a specific goal.
- **Core Modules:**
  1. **Perception:** Processing text, vision, and environmental cues.
  2. **Memory:** Retaining context over multi-turn interactions.
  3. **Action/Planning:** Breaking down tasks, utilizing tools, and executing code.

The Role of "Planning" in LLMs

Defining LLM Planning

Planning is the ability of an LLM to decompose a complex objective into manageable, actionable steps, execute them sequentially, and adjust based on feedback.

- Moves beyond single-turn QA.
- Requires *Reflection* (self-correction).
- Requires *Tool Utilization* (APIs, search engines, code interpreters).

Evolution: RAG vs. Agentic RAG

- **Traditional RAG:** Retrieves data from a vector database and generates a response based on that static retrieval.
- **Agentic RAG:** Embeds autonomous agents within the RAG framework.
- **Planning in Agentic RAG:** The agent dynamically decides *if* it needs to search, *what* to search for, evaluates the retrieved data, and iterates until the goal is met.

Key Attributes of LLM Planning Systems

- **Autonomy:** Minimal human intervention.
- **Adaptability:** Changing plans based on API errors or new data.
- **Multi-Step Reasoning:** Chain-of-Thought (CoT), Tree of Thoughts (ToT).
- **Tool Interaction:** GUI control, database querying.
- **Collaboration:** Multi-agent communication.

2. Architectures for LLM Planning

How do we structure an LLM to plan effectively?

- Single-Agent Architectures
- Multi-Agent Collaborative Systems
- Human-in-the-Loop (HITL) Systems

Single-Agent Planning Architecture

- **Designated Function:** Defines the agent's role (e.g., "Software Engineer").
- **Strategy Development (Planning):** Breaking down objectives into actionable steps.
- **Execution Environment:** Where tools (web search, calculators) are invoked.
- **Self-Evaluation:** Assessing the outcome. If unsatisfactory, the agent replans.

Agentic Workflows vs. Deterministic Workflows

Deterministic Workflow

Fixed, unchanging rules. If A happens, do B. Fails when edge cases occur.

Agentic Workflow

Dynamic and adaptive. The LLM acts as the routing engine, deciding the sequence of operations based on real-time context.

The Role of "Temperature" in LLM Planning

What is Temperature?

A hyperparameter that controls the randomness of the LLM's output. In the context of AI agents, it dictates the balance between **Exploration** (creativity) and **Exploitation** (determinism).

Low Temperature ($T \approx 0.0 - 0.2$)

- **Behavior:** Highly deterministic, focused, and repeatable.
- **Best For:** Executing rigid plans, Tool/API calling, mathematical reasoning, and factual retrieval.
- **Drawback:** May get stuck in a rigid mindset; poor at generating alternative plans if the first one fails.

High Temperature ($T \approx 0.7 - 1.0+$)

- **Behavior:** Creative, diverse, and unpredictable.
- **Best For:** Brainstorming, generating multiple diverse paths (e.g., Tree of Thoughts), and hypothesis generation.
- **Drawback:** High risk of hallucination; unreliable for exact JSON formatting or strict API execution.

Multi-Agent Systems (MAS)

- Harnesses the collective intelligence of multiple specialized agents.
- Simulates complex real-world environments through collaborative planning, discussion, and decision-making.
- **Coordination Topologies:** Star, Chain, Tree, and Graph-based communication.
- **Advantage:** Reduces the cognitive load on a single LLM by distributing tasks (e.g., one agent writes code, another tests it).

Overview of AI Agent Frameworks (2024-2025)

The paper highlights several major frameworks designed to facilitate LLM planning and agent creation:

- LangChain
- LlamaIndex
- CrewAI
- Swarm (OpenAI)
- OctoTools
- OpenAI Agents SDK

Framework: LangChain

- **Core Idea:** Integrates LLMs with diverse tools to build autonomous agents.
- **Planning Mechanism:** Uses agents that analyze a prompt, select appropriate tools for subtasks, and iteratively refine responses.
- **Example:** Parsing an email, deciding to book a calendar slot, checking availability via API, and confirming.

Framework: LlamaIndex

- **Core Idea:** Enables autonomous agent creation via external tool integration, heavily focused on data ingestion and routing.
- **Planning Mechanism:** Wraps functions into `FunctionTool` objects and employs a `ReActAgent` (Reason + Act) for stepwise tool selection.
- **Advantage:** Simplifies agent development with a dynamic, modular pipeline.

Framework: CrewAI

- **Core Idea:** Orchestrates teams of specialized AI agents for complex tasks.
- **Structure:**
  - *Crew* (oversight)
  - *Agents* (specialized roles like Researcher, Writer)
  - *Tasks* (assignments)
  - *Process* (collaboration planning)
- **Advantage:** Mimics human team collaboration with flexible, parallel workflows.

Framework: Swarm (OpenAI)

- **Core Idea:** A lightweight, stateless abstraction for multi-agent systems.
- **Planning Mechanism:** Dynamic handoffs. An agent can transfer control to another agent based on the conversation flow.
- **Advantage:** Fine-grained control, transparency, and compatibility with various backends (TGI, vLLM).

Framework: GUI Agents

- **Core Idea:** Facilitates computer control via natural language and visual inputs.
- **Planning Mechanism:** Translates user instructions and screenshots into desktop actions (cursor movements, clicks, keystrokes).
- **Example:** Claude 3.5 Computer Use. Capable of executing multi-step desktop workflows.

Framework: Agentic Reasoning & OctoTools

- **Agentic Reasoning:** Leverages specialized agents (Web-search, Coding, Mind Map) to iteratively refine multi-step reasoning.
- **OctoTools:** A training-free framework that combines standardized tool cards, a strategic planner, and an executor.
- **Performance:** Outperforms similar frameworks by up to 10.6% on complex reasoning tasks.

Human-in-the-Loop (HITL) Planning

- Autonomous planning can be dangerous in business-critical scenarios.
- **User Confirmation:** Planning pauses for user approval before executing "write" operations (e.g., deleting a database record).
- **Return of Control (ROC):** Elevates human involvement by returning action groups to the application layer, allowing users to edit parameters before execution.

Collaborative LLM Frameworks (TalkHier)

- **TalkHier:** A collaborative framework that integrates a rigorously structured messaging protocol.
- **Hierarchical Refinement:** Allows agents to iteratively improve their contributions at successive levels.
- **Benefit:** Mitigates biases introduced by sequential feedback and reduces misunderstandings in unstructured multi-agent exchanges.

3. Benchmarking LLM Planning and Reasoning

The Evaluation Challenge

Standard QA benchmarks (like MMLU) are no longer sufficient to test the complex planning, tool-use, and multi-step reasoning of modern AI Agents.

- We need dynamic, interactive, and multi-turn benchmarks.
- The survey covers 60 benchmarks developed between 2019 and 2025.

Taxonomy of LLM Benchmarks

Benchmarks are categorized into:

1. General & Academic Knowledge Reasoning
2. Mathematical Problem Solving
3. Code Generation & Software Engineering
4. Factual Grounding & Retrieval
5. Domain-Specific Evaluations (Med, Law, Cyber)
6. Multimodal & Embodied Tasks
7. Agentic & Interactive Assessments

Evaluating General Intelligence & Planning: GAIA

- **Benchmark:** GAIA (2024)
- **Focus:** General AI Assistants.
- **Task:** 466 curated questions requiring reasoning, multi-modality, web browsing, and tool use.
- **Key Finding:** Humans achieve 92% accuracy, while GPT-4 with plugins reaches only 15%. Highlights the gap in robust, general-purpose planning.

Evaluating Function Calling: ComplexFuncBench

- **Benchmark:** ComplexFuncBench (2025)
- **Focus:** Multi-step operations and tool selection.
- **Characteristics:** Evaluates reasoning over implicit parameters and constraints with input lengths up to 128k tokens.
- **Key Finding:** Significant limitations exist in current open models compared to closed models (Claude 3.5, GPT-4) regarding premature termination in multi-step planning.

Evaluating Hard Academic Reasoning: HLE

- **Benchmark:** Humanity's Last Exam (HLE, 2025)
- **Focus:** Expert-level academic reasoning.
- **Characteristics:** 3,000 questions spanning 100 subjects. Resistant to quick internet retrieval.
- **Key Finding:** State-of-the-art LLMs (DeepSeek R1, GPT-4o, Gemini Thinking) score below 10%, exposing severe limits in deep academic planning and logic.

Evaluating Multi-Agent Planning: MultiAgentBench

- **Benchmark:** MultiAgentBench (2025)
- **Focus:** Coordination and planning in dynamic environments.
- **Tasks:** Minecraft structure building, collaborative coding, competitive Werewolf gameplay.
- **Key Finding:** Direct peer-to-peer communication combined with cognitive planning yields a 3% improvement in milestone achievement. Graph-based protocols outperform others.

Evaluating Strategic Planning: SPIN-Bench

- **Benchmark:** SPIN-Bench (2025)
- **Focus:** Strategic Planning, Interaction, and Negotiation.
- **Tasks:** Classical PDDL tasks, competitive board games, cooperative card games.
- **Key Finding:** LLMs perform well on short-range planning but struggle with deep multi-hop reasoning and socially adept coordinated decision-making.

Evaluating Conversational Agents: τ-bench

- **Benchmark:** τ-bench (2024)
- **Focus:** Dynamic, multi-turn conversational planning.
- **Tasks:** Emulates real-world environments requiring domain-specific API use (e.g., booking flights, retail).
- **Key Finding:** SOTA function-calling agents succeed on less than 50% of tasks, showing marked inconsistency in rule-following and database state manipulation.

Evaluating Software Engineering Agents: SWE-Lancer

- **Benchmark:** SWE-Lancer (2025)
- **Focus:** Real-world freelance software engineering tasks.
- **Tasks:** 1,400 tasks from Upwork, ranging from minor bug fixes to feature implementations.
- **Key Finding:** Claude 3.5 Sonnet achieves only a 26.2% pass rate on independent tasks, indicating that autonomous coding and planning still require significant improvement.

Evaluating Mathematical Reasoning: ProcessBench

- **Benchmark:** ProcessBench (2024)
- **Focus:** Error detection in math reasoning steps.
- **Tasks:** 3,400 test cases. Models must identify the earliest erroneous step in a step-by-step solution.
- **Key Finding:** Evaluates Process Reward Models (PRMs). Open models like QwQ-32B-Preview show strong capabilities but fall short of reasoning-specialized models like o1-mini.

Evaluating Cybersecurity Planning: OCCULT

- **Benchmark:** OCCULT (2025)
- **Focus:** Operational evaluation of cybersecurity risks.
- **Tasks:** Simulates real-world threat scenarios to assess LLMs in offensive cyber operations.
- **Key Finding:** DeepSeek-R1 achieves over 90% in Threat Actor Competency Tests, showing advanced planning in adversarial environments.

Evaluating Factuality and RAG: FRAMES

- **Benchmark:** FRAMES (2024)
- **Focus:** Factuality, Retrieval, and Reasoning.
- **Tasks:** Multi-hop questions requiring integration of 2-15 Wikipedia articles.
- **Key Finding:** Multi-step retrieval pipelines (Agentic planning) improve accuracy from 40% to 66% compared to single-step models.

Evaluating Multimodal Reasoning: ENIGMAEVAL

- **Benchmark:** ENIGMAEVAL (2025)
- **Focus:** Complex multimodal puzzles.
- **Tasks:** 1,184 puzzles requiring synthesis of text and images, and multi-step deductive reasoning.
- **Key Finding:** SOTA systems score only 7% on standard puzzles, pushing models into highly unstructured, creative problem-solving scenarios.

Agent-as-a-Judge Evaluation Paradigm

- **Concept:** Using Agentic systems to evaluate other Agentic systems.
- **Why?** Traditional evaluation relies on fixed outcomes or manual labor.
- **How it works:** Provides granular, intermediate feedback throughout the task-solving process.
- **Results:** Achieves up to 90% alignment with human judgments while reducing evaluation costs by 97% compared to human assessment.

Summary of Benchmarks

- The trend is moving from **static Q&A** (MMLU) to **dynamic, multi-step environment evaluations** (GAIA, SWE-Lancer, τ-bench).
- Planning abilities are the primary bottleneck in current LLMs.
- Visual and embodied reasoning represent the next frontier in agent evaluation.

4. Applications of Agentic Planning

Where are these planning capabilities being deployed today?

- Software Engineering & Code Generation
- Healthcare & Clinical Diagnostics
- Scientific Discovery & Automated Research
- Finance & Market Simulation
- Multimedia Generation

Applications: Software Engineering (SE)

The Shift

From code autocomplete (Copilot) to autonomous software engineering (Agents that plan, write, test, and debug).

- **Agent Programming Architectures:** Treating LLMs as automata programmed via natural and formal languages (e.g., Ann Arbor Architecture).
- **Adaptive Control:** Systems like DARS (Dynamic Action Re-Sampling) allow coding agents to branch out at key decision points and recover from sub-optimal code.

SE Agents: Verification and Environment Setup

- **AgentGym & SWE-Gym:** Environments for training SWE agents on real-world repositories with execution feedback.
- **Repo2Run:** Automates the complex planning required for setting up Docker environments. Uses dual-environment rollback to prevent contamination from failed configurations.
- **LocAgent:** Uses graph-based representations of codebases for multi-hop code localization.

Applications: Healthcare & Clinical Support

- Healthcare requires highly reliable, explainable planning.
- **Chain-of-Diagnosis (CoD):** Transforms the diagnostic process into a transparent, step-by-step chain that mirrors a physician's reasoning.
- **ZODIAC:** A multi-agent system for cardiological diagnostics. Agents extract features, detect arrhythmias, and draft reports collaboratively.

Healthcare: Differential Diagnosis

- **MEDDxAgent:** Facilitates interactive, iterative diagnostic reasoning rather than relying on a static patient profile.
- **PathFinder:** Multi-agent WSI (Whole-Slide Image) analysis. Includes a Triage Agent, Navigation Agent, Description Agent, and Diagnosis Agent.
- *Planning in action:* The Navigation agent plans where to zoom in, while the Diagnosis agent synthesizes the findings.

Healthcare: Mental Health & Therapy Agents

- **Scripted Therapy Agents:** Constrain LLM responses via expert-written scripts. The agent plans its responses by transitioning through finite conversational states.
- **CAMI:** Grounded in Motivational Interviewing. Uses modules for State inference, Topic exploration, and Response generation.
- **PsyDraw:** Multi-agent system analyzing House-Tree-Person (HTP) drawings for psychological screening.

Healthcare: Drug Discovery & Inpatient Care

- **LIDDiA:** Autonomous in-silico agent that orchestrates end-to-end drug discovery, from target selection to lead optimization.
- **DrugAgent:** Multi-agent framework for drug repurposing (integrates Knowledge Graphs and Literature mining).
- **MAP Framework:** Multi-Agent Inpatient Pathways. A Chief Agent coordinates a Triage Agent, Diagnosis Agent, and Treatment Agent for complex care planning.

Applications: Scientific Discovery

Agentic Science

LLM agents are now generating hypotheses, conducting literature reviews, and designing experiments.

- **AgentRxiv:** Enables autonomous LLM agent laboratories to share preprints and build upon each other's research iteratively.
- **AI Co-Scientist (Google):** A multi-agent system generating novel research hypotheses, utilizing simulated debates and Elo ranking tournaments.

Science: Literature Review & Ideation

- **SurveyX:** Decomposes survey composition into Preparation (retrieval + structuring) and Generation (repolishing) phases.
- **Chain-of-Ideas (CoI) Agent:** Structures literature into a progressive chain to capture current advancements and generate innovative research ideas.
- **Data Interpreter:** Manages end-to-end data science workflows via Hierarchical Graph Modeling to decompose complex subproblems.

Science: Materials & Biomedical Research

- **HoneyComb:** LLM-agent for materials science. Uses an Inductive Tool Construction method to generate and refine API tools for querying material databases.
- **GeneAgent:** Features self-verification capabilities to interact with biological databases, minimizing hallucinations in functional genomics.
- **PRefLexOR:** Uses recursive learning and RL to self-improve reasoning steps during biomedical exploration.

Applications: Mathematical Problem Solving

- Math requires structured reasoning, formal logic, and precise numerical computation.
- **MACM Prompting:** Multi-Agent System for Conditional Mining. Iteratively refines complex multi-step math problems.
- **MathLearner:** Enhances reasoning via an inductive retrieval loop, applying prior knowledge to new tasks.

Math: Theorem Proving and Formalization

- **MA-LoT:** Multi-agent framework for Lean4 theorem proving. Synergizes natural-language reasoning with formal verification feedback.
- **KG-Proof Agent:** Augments LLMs with knowledge graphs to structure lemmas and formalize mathematical proofs.
- **Flows:** Uses online Direct Preference Optimization (DPO) where component LLMs collaboratively build coherent reasoning traces.

Math: Tutoring and Visual Reasoning

- **PACE:** Personalized Conversational Tutoring Agent. Simulates distinct student personas and adapts teaching strategies dynamically.
- **Agent Trading Arena:** Evaluates numerical reasoning on visual data (scatter plots, K-line charts) rather than just plain text, improving geometric reasoning.

Applications: Finance & Market Simulation

- **TwinMarket:** Simulates complex socio-economic systems. Harnesses LLM cognitive biases to model behavioral economics (e.g., market bubbles).
- **FinCon:** Models a manager-analyst communication hierarchy for sequential investment decision-making. Features a self-critiquing risk-control module.

Finance: Strategic Behavior & Analysis

- **Competitive Markets:** LLM agents independently engage in strategic behaviors like collusion or market division (Cournot competition).
- **MarketSenseAI:** RAG-based multi-agent system that processes SEC filings, earnings calls, and news for comprehensive stock analysis.
- **FinSphere:** Conversational stock analysis agent utilizing real-time data feeds and quantitative tools.

Applications: Multimedia Generation

- **FilmAgent:** Automates 3D virtual film production. Agents act as directors, screenwriters, actors, and cinematographers, iterating on feedback.
- **AesopAgent:** Story-to-video production pipeline using an evolutionary workflow to optimize prompts and visual coherence.
- **IBSEN:** Drama script generation. A director agent guides actor agents to role-play and adjust narrative plots.

Multimedia: Music & Lyric Generation

- **ComposerX:** Multi-agent symbolic music generation. Agents specialize in melody, harmony, and structure while respecting long-term dependencies.
- **Lyric Generation Agents:** Decomposes melody-to-lyric alignment in tonal languages (like Mandarin) into sub-tasks (rhyme, syllable count, consistency).
- **MusicAgent:** Orchestrates diverse music processing tools via an autonomous LLM workflow.

Synthetic Data Generation via Agents

AgentInstruct

Uses multi-agent workflows to automate the creation of high-quality instructional datasets (Generative Teaching).

- Transforms raw text/code into refined instruction pairs via suggester-editor agent pairs.
- Generated a 25 million prompt-response dataset, significantly improving models like Mistral-7B.

Geographic Information Systems (GIS)

- **Autonomous GIS Agent:** Utilizes LLMs to perform spatial analyses and cartographic tasks.
- **Planning aspect:** Autonomously discovers, retrieves, and writes code to process geospatial data from sources like OpenStreetMap and ESRI.
- **MineAgent:** Multi-modal agent for remote-sensing mineral exploration, using hierarchical judging for spatial-spectral integration.

Summary of Applications

- Across all domains, the unifying theme is **Task Decomposition**.
- Complex problems (drug discovery, film making, software engineering) are solved by creating a *plan*, assigning roles to *agents*, and executing iteratively.

5. Agent Communication Protocols

For agents to plan and act in the real world, they need standardized ways to talk to tools, data, and other agents.

- MCP (Model Context Protocol)
- ACP (Agent Communication Protocol)
- A2A (Agent-to-Agent Protocol)

Model Context Protocol (MCP) by Anthropic

- **Goal:** A "USB-C" port for AI. Standardizes how AI systems interact with external tools and data sources.
- **Architecture:** Client-server model using JSON-RPC 2.0.
- **Use Case:** An LLM securely accessing local files, PostgreSQL databases, or GitHub repositories uniformly.
- **State Management:** Host-mediated stateful sessions that aggregate conversation history.

Agent Communication Protocol (ACP) by IBM

- **Goal:** Standardize communication between agents within an experimental platform (BeeAI).
- **Architecture:** Local-first orchestration. Messages are custom JSON objects handling MIME types and metadata.
- **Use Case:** Running and managing various specialized agents within a unified local environment, tracking telemetry and agent discovery.

Agent-to-Agent Protocol (A2A) by Google

- **Goal:** Interoperability among agents across different frameworks (LangChain, CrewAI, AutoGen, etc.).
- **Architecture:** Uses HTTP, Server-Sent Events (SSE), and JSON-RPC.
- **Use Case:** Enabling dynamic collaboration between autonomous agents from diverse vendors without sharing internal memories or resources.

Comparing Protocols for Planning Integration

Feature	MCP	ACP	A2A
Primary Focus	Tool & Data Integration	Internal Agent Orchestration	Cross-Framework Agent Collab
Architecture	Client-Server (JSON-RPC)	Local BeeAI Server	HTTP / SSE interfaces
Use Case	LLM querying a local DB	Multi-agent workflow in BeeAI	LangChain agent talks to CrewAI

Multi-Agent Integration Framework

- Modern architectures combine these protocols.
- **MCP** connects the agents to the data layer (Tools, APIs).
- **A2A** connects the agents to each other (CrewAI agent delegating a task to a Haystack agent).
- This creates a highly scalable environment for complex, multi-agent planning.

6. Challenges and Future Directions

Despite rapid progress, LLM planning and autonomous agents face critical hurdles:

- Limits of Agentic Reasoning
- Multi-Agent Failure Modes
- Tool Integration Scaling
- Security and Vulnerabilities

Challenges in Agentic Reasoning

- **The Meta-CoT Challenge:** Traditional Chain-of-Thought reveals final reasoning steps but not the *latent cognitive process* (the search and iteration).
- **Need for RL:** Moving forward requires training pipelines that integrate instruction tuning with linearized search traces and Reinforcement Learning (RL) post-training.
- **Balancing Act:** Exploration of novel tool sequences vs. Exploitation of known effective patterns.

Why Do Multi-Agent Systems Fail?

A rigorous study of 150 tasks across 5 frameworks identified 14 distinct failure modes, grouped into three categories:

1. **Design Shortcomings:** Ignoring task/role specifications, unnecessary repetitions, memory lapses.
2. **Inter-agent Misalignment:** Agents failing to agree or understand the outputs of other agents.
3. **Verification & Termination:** Inability to recognize when a plan is successfully completed, leading to infinite loops.

Scaling Automated Scientific Discovery

- **The Goal:** Fully autonomous AI scientists.
- **The Challenge:** Navigating the risk of producing biased or spurious hypotheses without human oversight.
- **Data Contamination:** Extracting truly innovative insights is hard when recent publications overlap with the LLM's pre-training data.

Dynamic Tool Integration at Scale

- **Current State:** Agents work well with a predefined, small set of tools.
- **The Challenge:** How to scale to massive, unseen tool pools (e.g., thousands of APIs).
- **Chain-of-Tools:** An approach that incorporates tool calling directly into the reasoning chain, but managing massive prompt-based demonstrations remains highly inefficient.

Security Vulnerabilities in Agent Protocols

The Risk of Autonomy

Autonomous planning means autonomous execution.

- **Protocol Risks (MCP/A2A):** Decentralized design lacks a central security authority.
- **Exploits:** Prompt injections can lead to protocol exploits (e.g., an agent tricked into deleting files via an MCP file server).
- **Need:** Standardized authentication mechanisms and robust logging/debugging for multi-step distributed workflows.

Future Directions

- **Inference-Time Scaling:** Spending more compute during the planning phase (like OpenAI o1/o3 and DeepSeek-R1) rather than just pre-training.
- **Domain-Specific Optimizations:** Smaller, highly specialized models (e.g., DeepSeek-R1-Distill) achieving optimal cost-accuracy trade-offs for specific workflows.
- **Improved Verifiers:** Training specialized reward models to grade intermediate planning steps.

Conclusion

- LLMs have evolved from conversational chatbots to autonomous agents capable of complex planning and execution.
- Frameworks (LangChain, CrewAI) and Protocols (MCP, A2A) are standardizing how agents interact.
- While applications in SE, Healthcare, and Science are booming, robust evaluation (benchmarks) and safety/security guardrails remain the most pressing open problems.

Thank You!

Questions & Discussion